

Universidad Mariano Gálvez de Guatemala



Curso: Programación III

Tema: Tarea XI tabla hash

Docente: Inge. David Alvarez

Alumno: Carlos Daniel Castro Caceres

```
Bienvenido  main.cpp 2 x  estudiantes.csv
cpp-hash > G+ main.cpp > main()
1  #include <iostream>
2  #include <fstream>
3  #include <sstream>
4
5  using namespace std;
6
7  // =====
8  // ESTRUCTURA STUDENT
9  // =====
10
11  struct Student {
12
13      int student_id;
14      string full_name;
15      string career;
16      int semester;
17      float gpa;
18      int skill_score;
19  };
20
21
22  // =====
23  // NODO PARA LISTA ENLAZADA
24  // =====
25
26  struct Node {
27
28      Student data;
29      Node* next;
30  };
31
32
```

```
32
33  // =====
34  // HASH TABLE
35  // =====
36
37  class HashTable {
38
39  private:
40
41      static const int SIZE = 10;
42
43      Node* table[SIZE];
44
45      int totalStudents;
46      int collisions;
47
48  public:
49
50      HashTable() {
51
52          totalStudents = 0;
53          collisions = 0;
54
55          for(int i = 0; i < SIZE; i++) {
56              table[i] = NULL;
57          }
58      }
59
60      int hashFunction(int id) {
61          return id % SIZE;
62      }
63  }
```

```

62     }
63
64     void insert(Student student) {
65
66         int index = hashFunction(student.student_id);
67
68         Node* temp = table[index];
69
70         while(temp != NULL) {
71
72             if(temp->data.student_id == student.student_id) {
73                 cout << "Estudiante duplicado: "
74                 << student.student_id << endl;
75                 return;
76             }
77
78             temp = temp->next;
79         }
80
81         Node* newNode = new Node();
82         newNode->data = student;
83         newNode->next = NULL;
84
85         if(table[index] != NULL) {
86             collisions++;
87             newNode->next = table[index];
88         }
89
90         table[index] = newNode;
91         totalStudents++;

```

```

91         totalStudents++;
92     }
93
94     void search(int id) {
95
96         int index = hashFunction(id);
97         Node* temp = table[index];
98
99         while(temp != NULL) {
100
101             if(temp->data.student_id == id) {
102
103                 cout << "\nESTUDIANTE ENCONTRADO\n";
104                 cout << "ID: " << temp->data.student_id << endl;
105                 cout << "Nombre: " << temp->data.full_name << endl;
106                 cout << "Carrera: " << temp->data.career << endl;
107                 cout << "Semestre: " << temp->data.semester << endl;
108                 cout << "GPA: " << temp->data.gpa << endl;
109                 cout << "Skill Score: " << temp->data.skill_score << endl;
110
111                 return;
112             }
113
114             temp = temp->next;
115         }
116
117         cout << "\nEstudiante no encontrado\n";
118     }
119
120     void remove(int id) {

```

```

120 void remove(int id) {
121
122     int index = hashFunction(id);
123
124     Node* temp = table[index];
125     Node* prev = NULL;
126
127     while(temp != NULL && temp->data.student_id != id) {
128         prev = temp;
129         temp = temp->next;
130     }
131
132     if(temp == NULL) {
133         cout << "\nEstudiante no encontrado\n";
134         return;
135     }
136
137     if(prev == NULL) {
138         table[index] = temp->next;
139     } else {
140         prev->next = temp->next;
141     }
142
143     delete temp;
144     totalStudents--;
145
146     cout << "\nEstudiante eliminado correctamente\n";
147 }
148
149 void display() {
150

```

```

150
151     cout << "\n==== TABLA HASH =====\n";
152
153     for(int i = 0; i < SIZE; i++) {
154
155         cout << "\nBucket " << i << ": ";
156
157         Node* temp = table[i];
158
159         while(temp != NULL) {
160
161             cout << "[" << temp->data.student_id
162                 << " - "
163                 << temp->data.full_name
164                 << "]" -> ";
165
166             temp = temp->next;
167         }
168
169         cout << "NULL";
170     }
171
172     cout << endl;
173 }
174
175 void statistics() {
176
177     float loadFactor = (float) totalStudents / SIZE;
178
179     cout << "\n==== ESTADISTICAS =====\n";

```

```

179         cout << "\n===== ESTADISTICAS =====\n";
180         cout << "Total estudiantes: " << totalStudents << endl;
181         cout << "Colisiones: " << collisions << endl;
182         cout << "Factor de carga: " << loadFactor << endl;
183     }
184 };
185
186
187
188 void loadCSV(string filename, HashTable& hashTable) {
189     ifstream file(filename);
190
191     if(!file.is_open()) {
192         cout << "Error al abrir el archivo: " << filename << endl;
193         return;
194     }
195
196     string line;
197
198     getline(file, line);
199
200     while(getline(file, line)) {
201         stringstream ss(line);
202         Student s;
203         string temp;
204
205         getline(ss, temp, ',');
206         s.student_id = stoi(temp);

```

```

208         s.student_id = stoi(temp);
209
210         getline(ss, s.full_name, ',');
211         getline(ss, s.career, ',');
212
213         getline(ss, temp, ',');
214         s.semester = stoi(temp);
215
216         getline(ss, temp, ',');
217         s.gpa = stof(temp);
218
219         getline(ss, temp, ',');
220         s.skill_score = stoi(temp);
221
222         hashTable.insert(s);
223     }
224
225     file.close();
226 }
227
228 // =====
229 // MAIN
230 // =====
231
232 int main() {
233
234     cout << "Nombre: TU NOMBRE: Carlos Daniel Castro Caceres\n";
235     cout << "Carné: TU CARNE: 9941-24-10004\n";
236
237     HashTable hashTable;

```

```

237     HashTable hashTable;
238
239
240     loadCSV("./estudiantes.csv", hashTable);
241
242     cout << "\nESTUDIANTES CARGADOS\n";
243
244     hashTable.display();
245     hashTable.statistics();
246
247     hashTable.search(1002);
248     hashTable.remove(1003);
249
250     hashTable.display();
251     hashTable.statistics();
252
253     return 0;
254 }

```

```

Nombre: TU NOMBRE: Carlos Daniel Castro Caceres
Carn|@: TU CARNE: 9941-24-10004

```

ESTUDIANTES CARGADOS

===== TABLA HASH =====

```

Bucket 0: NULL
Bucket 1: [1001 - Juan Perez] -> NULL
Bucket 2: [1002 - Ana Lopez] -> NULL
Bucket 3: [1003 - Carlos Ruiz] -> NULL
Bucket 4: [1004 - Maria Gomez] -> NULL
Bucket 5: [1005 - Pedro Martinez] -> NULL
Bucket 6: NULL
Bucket 7: NULL
Bucket 8: NULL
Bucket 9: NULL

```

===== ESTADISTICAS =====

```

Total estudiantes: 5
Colisiones: 0
Factor de carga: 0.5

```

ESTUDIANTE ENCONTRADO

```

ID: 1002
Nombre: Ana Lopez
Carrera: Ingenieria Industrial
Semestre: 7
GPA: 91.2
Skill Score: 200

```

Estudiante eliminado correctamente

===== TABLA HASH =====

```

Bucket 0: NULL
Bucket 1: [1001 - Juan Perez] -> NULL
Bucket 2: [1002 - Ana Lopez] -> NULL
Bucket 3: NULL
Bucket 4: [1004 - Maria Gomez] -> NULL
Bucket 5: [1005 - Pedro Martinez] -> NULL
Bucket 6: NULL
Bucket 7: NULL
Bucket 8: NULL
Bucket 9: NULL

```

===== ESTADISTICAS =====

```

Total estudiantes: 4
Colisiones: 0
Factor de carga: 0.4

```

Main.java

```
java-hashmap > J Main.java
1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.util.HashMap;
4
5  public class Main {
6
7      public static void main(String[] args) {
8
9          System.out.println("Nombre: Carlos Daniel Castro Caceres");
10         System.out.println("Carne: 9941-24-10004");
11
12         HashMap<Integer, Estudiante> estudiantes =
13             new HashMap<>();
14
15         cargarCSV("estudiantes.csv", estudiantes);
16
17         System.out.println("\nTOTAL ESTUDIANTES: "
18             + estudiantes.size());
19
20         System.out.println("\n===== HASHMAP =====");
21
22         for(Integer key : estudiantes.keySet()) {
23
24             System.out.println(
25                 "\nClave: " + key);
26
27             System.out.println(
28                 estudiantes.get(key));
29         }
30
31         buscar(estudiantes, 1002);
32
```

```
32         buscar(estudiantes, 1002);
33
34         eliminar(estudiantes, 1003);
35
36         System.out.println(
37             "\nTOTAL DESPUES DE ELIMINAR: "
38             + estudiantes.size());
39
40     }
41
42     public static void cargarCSV(
43         String archivo,
44         HashMap<Integer, Estudiante> estudiantes
45     ) {
46
47         try {
48
49             BufferedReader br =
50                 new BufferedReader(
51                     new FileReader(archivo));
52
53             String linea;
54
55             br.readLine();
56
57             while((linea = br.readLine()) != null) {
58
59                 String[] datos =
60                     linea.split(",");
61
```

```

61
62         int id =
63             Integer.parseInt(datos[0]);
64
65         if(estudiantes.containsKey(id)) {
66
67             System.out.println(
68                 "Duplicado: " + id);
69
70             continue;
71
72         }
73
74         Estudiante e =
75             new Estudiante(
76                 id,
77                 datos[1],
78                 datos[2],
79                 Integer.parseInt(datos[3]),
80                 Double.parseDouble(datos[4]),
81                 Integer.parseInt(datos[5])
82             );
83
84         estudiantes.put(id, e);
85
86     }

```

```

86     }
87
88     br.close();
89
90 } catch(Exception e) {
91
92     System.out.println(
93         "Error: " + e.getMessage());
94
95 }
96
97 }
98
99 public static void buscar(
100     HashMap<Integer, Estudiante> estudiantes,
101     int id
102 ) {
103
104     System.out.println(
105         "\n===== BUSQUEDA =====");
106
107     if(estudiantes.containsKey(id)) {
108
109         System.out.println(
110             estudiantes.get(id));
111
112     } else {
113

```



```

113     } else {
114         System.out.println(
115             "Estudiante no encontrado");
116     }
117 }
118
119 }
120
121 public static void eliminar(
122     HashMap<Integer, Estudiante> estudiantes,
123     int id
124 ) {
125
126     System.out.println(
127         "\n==== ELIMINACION =====");
128
129     if(estudiantes.containsKey(id)) {
130
131         estudiantes.remove(id);
132
133         System.out.println(
134             "Estudiante eliminado");
135     } else {
136
137         System.out.println(
138             "Estudiante no encontrado");
139     }
140 }
141

```

```

141     }
142
143 }
144
145 }

```

Estudiante.java

java-hashmap > J Estudiante.java

```
1  public class Estudiante {
2
3      int student_id;
4      String full_name;
5      String career;
6      int semester;
7      double gpa;
8      int skill_score;
9
10     public Estudiante(
11         int student_id,
12         String full_name,
13         String career,
14         int semester,
15         double gpa,
16         int skill_score
17     ) {
18
19         this.student_id = student_id;
20         this.full_name = full_name;
21         this.career = career;
22         this.semester = semester;
23         this.gpa = gpa;
24         this.skill_score = skill_score;
25
26     }
27
28     public String toString() {
29
30         return "ID: " + student_id +
31             "\nNombre: " + full_name +
32             "\nCarrera: " + career +
```

```
32         "\nCarrera: " + career +
33         "\nSemestre: " + semester +
34         "\nGPA: " + gpa +
35         "\nSkill Score: " + skill_score;
36
37     }
38
39 }
```

Nombre: Carlos Daniel Castro Caceres
Carne: 9941-24-10004

TOTAL ESTUDIANTES: 5

===== HASHMAP =====

Clave: 1001
ID: 1001
Nombre: Juan Perez
Carrera: Ingenieria en Sistemas
Semestre: 5
GPA: 85.5
Skill Score: 150

Clave: 1002
ID: 1002
Nombre: Ana Lopez
Carrera: Ingenieria Industrial
Semestre: 7
GPA: 91.2
Skill Score: 200

Clave: 1003
ID: 1003
Nombre: Carlos Ruiz
Carrera: Arquitectura
Semestre: 3
GPA: 78.0
Skill Score: 120

Clave: 1004
ID: 1004
Nombre: Maria Gomez

Nombre: Maria Gomez
Carrera: Derecho
Semestre: 8
GPA: 88.7
Skill Score: 180

Clave: 1005
ID: 1005
Nombre: Pedro Martinez
Carrera: Medicina
Semestre: 10
GPA: 95.1
Skill Score: 250

===== BUSQUEDA =====

ID: 1002
Nombre: Ana Lopez
Carrera: Ingenieria Industrial
Semestre: 7
GPA: 91.2
Skill Score: 200

===== ELIMINACION =====

Estudiante eliminado

TOTAL DESPUES DE ELIMINAR: 4